

HUGGING FACE & STANFORD

SmoVLM: Redefining Small and Efficient Multimodal Models

Competitive Performance at Sub-1GB Memory Footprint

Large VLMs Are Powerful but Impractical for Edge Deployment

The Memory Bottleneck

- Current state-of-the-art VLMs require massive compute and memory footprints.
- Most edge devices (phones, IoT) have strict memory budgets (<4GB available for ML).
- SOTA models like Molmo-7B require **27.7GB RAM**, limiting mobile deployment.

TRADITIONAL SOTA VLM

Molmo-A1B-7B

Params: **7.2 Billion** RAM Need: **27.7 GB**

Result: Cloud-dependent, high latency

VS

SMOLVLM TARGET

SmoIVLM-256M

Params: **256 Million** RAM Need: **< 1.0 GB**

Result: Real-time on-device inference

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2

Image Splitting

Processes high-resolution images by splitting them into smaller patches.

Vision Encoder

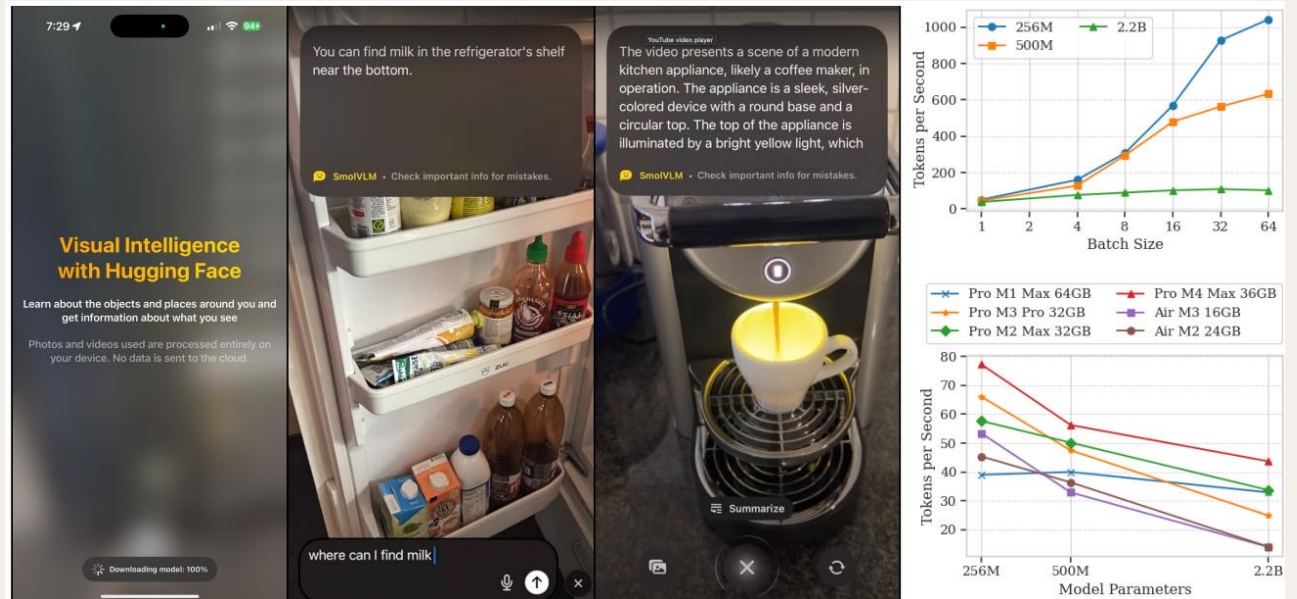
Extracts rich visual features using the highly efficient SigLIP-B/16.

Pixel Shuffle (r=4)

Aggressively compresses tokens by 16× to drastically reduce sequence length.

Linear Projection & SmolLM2

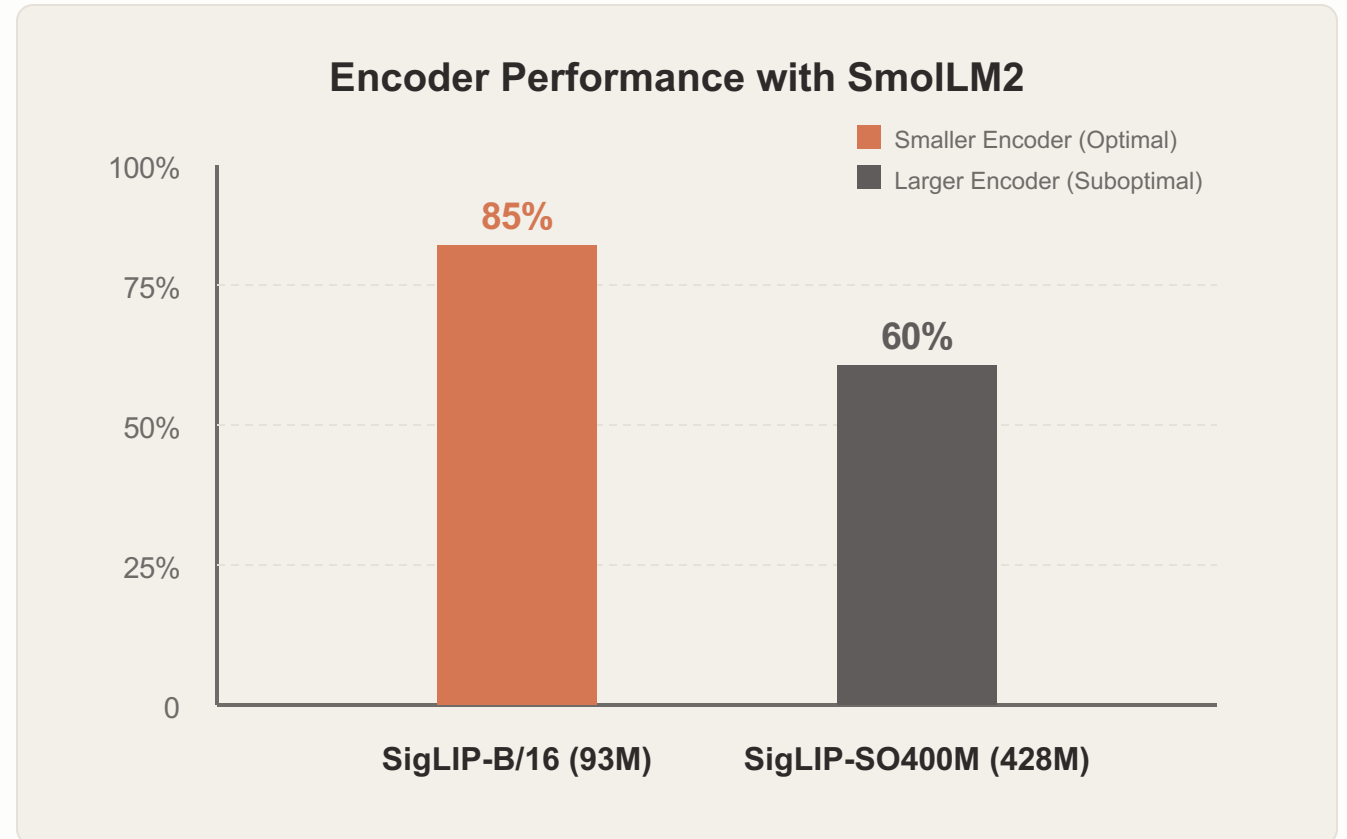
Maps visual features directly to the SmolLM2 language model space.



Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

🔧 Finding 1: Balance is Key

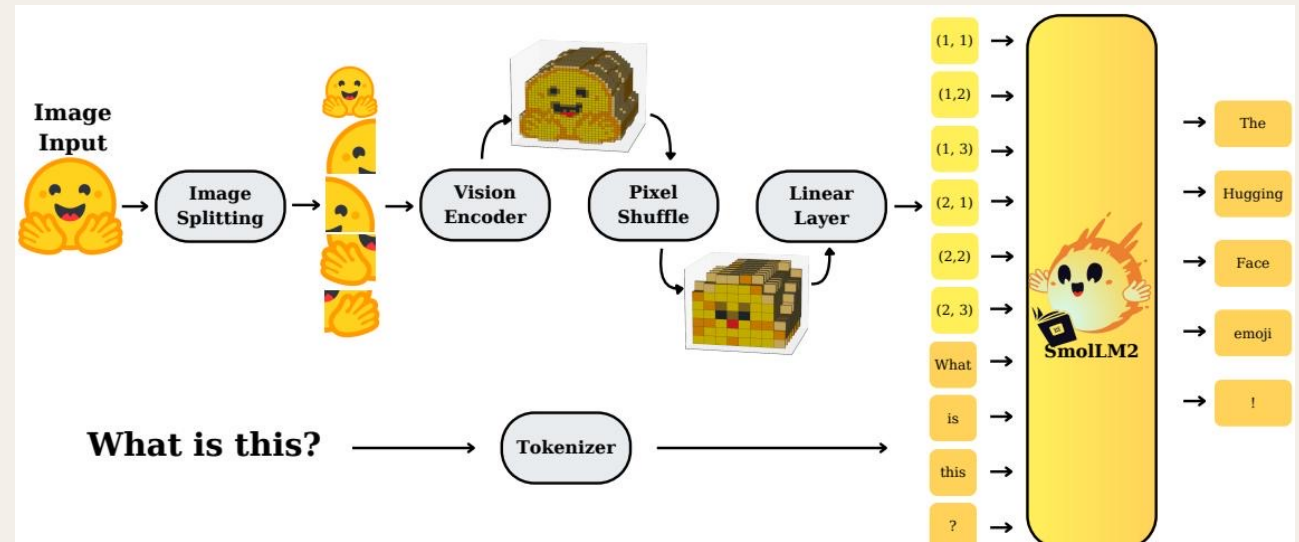
- Balanced encoder-LM parameter allocation matters.
- For smaller LMs (like SmoLLM2), pairing with a smaller vision encoder
- **yields better performance** Why? It ensures the language model has enough capacity to reason over the extracted visual features.



Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16×

↓ Token Compression

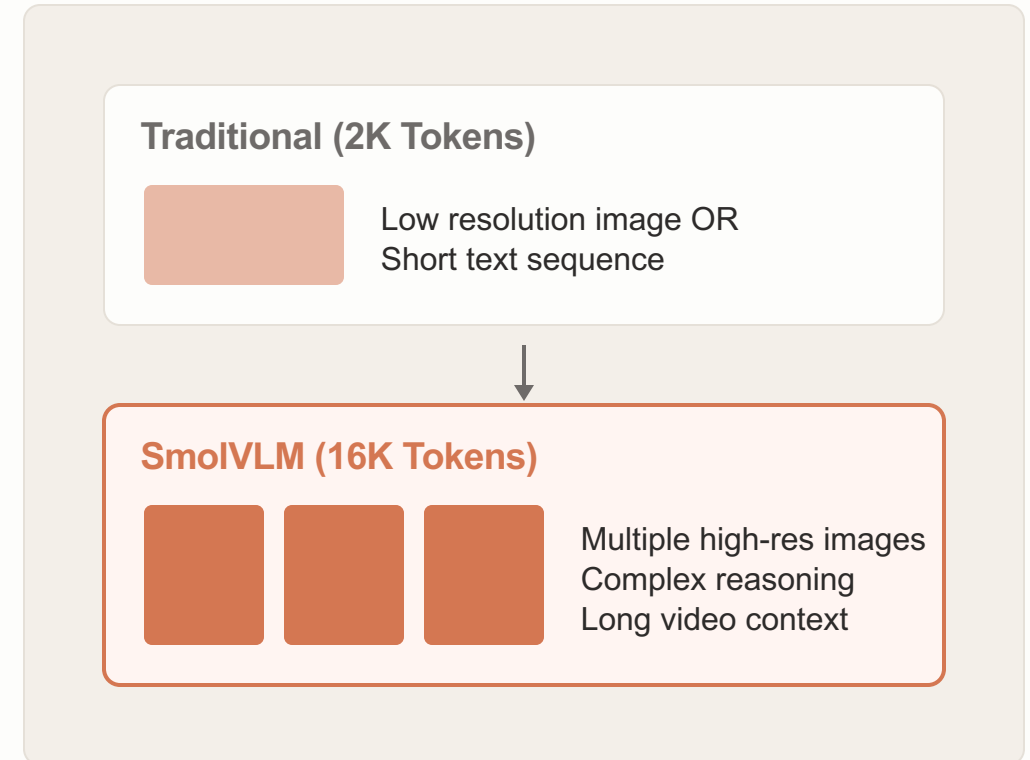
- Pixel shuffle rearranges spatial tokens to reduce sequence length.
- A shuffle factor of $r=4$ reduces **visual tokens by 16×**.
- Small VLMs benefit immensely from $r=4$ compression, whereas larger models typically prefer $r=2$.
- Outcome: Processing multiple images or longer videos within limited context windows.



Extending Context from 2K to 16K Tokens Unlocks Higher Resolution

↔ Finding 2: Context Scaling

- Performance improves significantly with extended context.
- RoPE base adjustment: Scaled from 10K to 273K for better long-context.
- Training strategy: Fine-tuned on mixed long and short-context data.
- The 2.2B model utilizes **16K context**, enabling high-resolution multi-image reasoning.



SmoIVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

 Comparison against Best Efficient OS baselines across Single-Image, Multi-task, and Video benchmarks

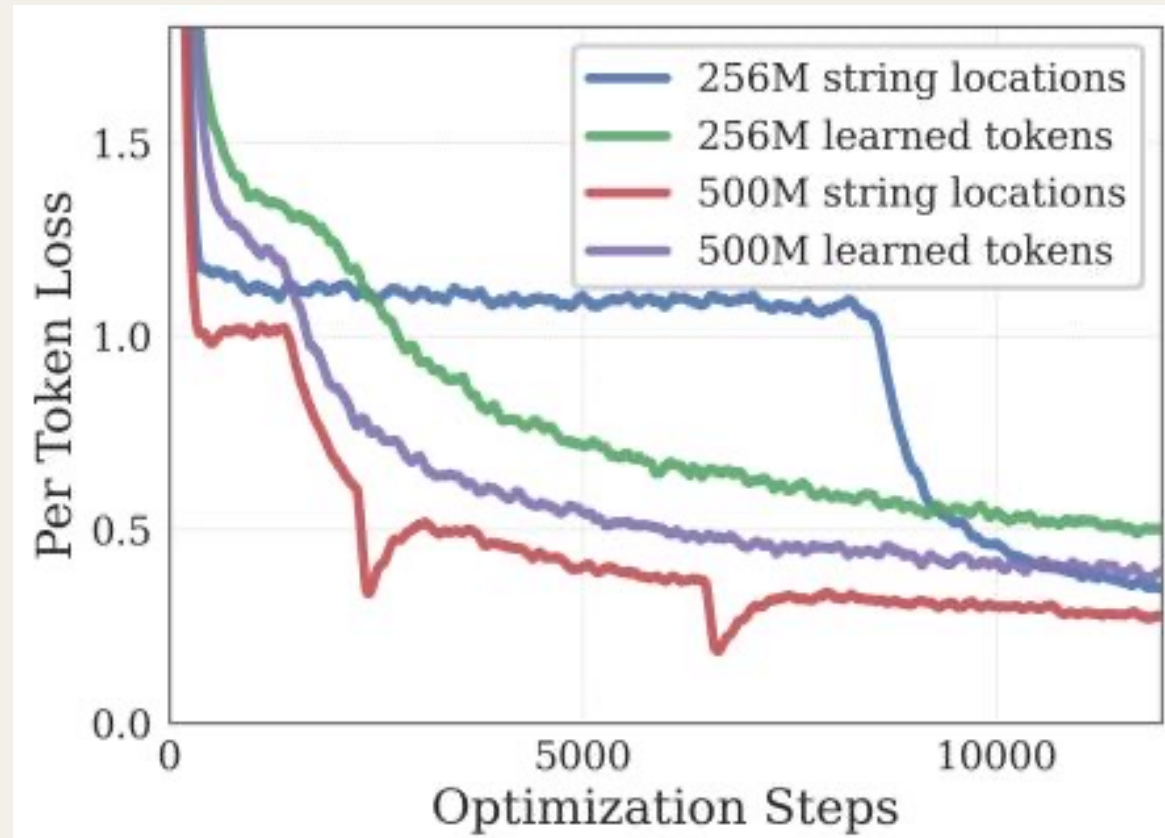
Model	Parameters	RAM Usage	Average Score
Molmo-A1B-7B (SOTA)	7.2B	27.7 GB	-
InternVL2-2B	2.0B	-	-
SmoIVLM-2.2B	2.2B	4.9 GB	59.8%
SmoIVLM-500M	500M	1.5 GB	51.0%

Note: SmoIVLM-256M achieves 44.0% average score at just**0.8 GB RAM**, surpassing the 300× larger Idefics-80B.

Real-Time Inference on iPhones: Up to 80 Tokens/Second

Local Execution on iOS

- HuggingSnap app demonstrates fully local execution on iPhones.
- Runs efficiently on Apple Silicon via the MLX framework.
- Throughput reaches up to **80 tokens/second** depending on model size and batching strategy.
- Proves the viability of sub-1GB VLMs for interactive edge applications without cloud dependencies.



Key Takeaways: Design Principles for Efficient Multimodal Models

01

Balance Compute

Balance encoder-LM compute allocation. Smaller models benefit from proportionally smaller vision encoders to save capacity for reasoning.

02

Extend Context

Extend context length (16K+) to unlock higher image resolution and multi-image processing capabilities even on edge devices.

03

Aggressive Compression

Use aggressive token compression ($r=4$) via pixel shuffle for small models. Reduces visual tokens by 16 $\times$ with minimal performance drop.

04

On-Device Viability

Sub-1GB inference is practically achievable. Models like SmolVLM-256M deliver competitive multimodal AI **fully offline on smartphones.**